A Symbolic Constraint Satisfaction Solver for Zebra Logic Puzzles

1. Introduction

Logic grid puzzles, commonly referred to as Zebra puzzles, are a classical class of problems in artificial intelligence that can be naturally modeled as Constraint Satisfaction Problems (CSPs). These puzzles require assigning values to variables such that a set of logical constraints is satisfied. The **AI Connect 2025 Challenge** evaluates CSP solvers on their ability to correctly and efficiently solve a large collection of such puzzles drawn from the ZebraLogicBench dataset.

This project presents a **purely symbolic CSP solver** for Zebra logic puzzles. The solver emphasizes correctness and transparency by exhaustively enumerating candidate solutions and systematically eliminating those that violate puzzle constraints. Its performance is evaluated using accuracy and step-based efficiency metrics defined by the challenge.

---

2. Problem Formulation

Each Zebra puzzle is modeled as a CSP defined by the tuple *(Variables, Domains, Constraints)*:

- **Variables:**
  Each attribute–value pair (e.g., `Name_Alice`, `Color_Red`) corresponds implicitly to a position (house index) within a complete puzzle grid.
- **Domains:**
  Variable domains are represented implicitly through complete candidate grids. For a puzzle with *N* houses and fixed attribute sets, each candidate grid assigns exactly one house index to each attribute–value.
- **Constraints:**
  Constraints are derived from both puzzle structure and natural language clues:
  - Uniqueness constraints ensure that each attribute value appears exactly once per grid.
  - Positional constraints represent fixed or relative positions.
  - Relational constraints encode equality, adjacency, ordering, and distance relationships (e.g., “left of”, “next to”).

Rather than incrementally constructing assignments, the solver evaluates full candidate grids against these constraints.

---

3. System Architecture

The system is organized into four main components:

1. **Data Parsing Module (`parser.py`)**
   Reads puzzle text and converts each natural-language clue into a constraint-checking function applicable to candidate grids.
2. **CSP Solver Core (`solver.py`)**
   Generates all possible complete grids for a puzzle and eliminates those that violate one or more constraints.
3. **Execution Script (`run.py`)**
   Runs the solver over the dataset and produces the final `results.csv` file.
4. **Evaluation Output**
   Collects per-puzzle solutions and step counts used for scoring.

This modular structure ensures a clear separation between parsing, solving logic, and evaluation.

---

4. Solving Approach

Instead of traditional incremental backtracking, the solver adopts a **candidate elimination strategy**:

1. **Candidate Generation**
   All possible complete grids consistent with the puzzle size and attributes are generated.
   For example, a 3×3 puzzle yields 216 valid permutations.
2. **Constraint Propagation (Forward Checking)**
   Each clue is applied to the candidate set. Any grid that violates a constraint is immediately discarded.
   Each elimination counts as a *forward-checking step* if it reduces the remaining candidate set.
3. **Decision Steps**
   If multiple valid candidates remain after all constraints are applied, the puzzle is under-specified.
   The solver selects one valid solution and counts the number of necessary choices as *decision steps*.

The total step count is defined as:

$$
\text{steps} = \text{forward_check_steps} + \text{decision_steps}
$$

This approach remains fully symbolic and aligns with CSP principles through variables, domains, and constraint-based elimination.

---

5. Step Counting and Solver Traces

Although the solver does not build an explicit search tree, it records step counts reflecting logical effort:

- **Forward-checking steps:**
  Constraint applications that eliminate at least one candidate grid.
- **Decision steps:**
  Logical choices required when multiple valid solutions remain.

These metrics provide an interpretable notion of solver effort and are included in the final output.

---

6. Experimental Setup

- **Dataset:** ZebraLogicBench (~1,000 puzzles)
- **Execution Environment:**
  - CPU-only execution
  - Single-threaded Python implementation
- **Team Size:** 7 members

The solver supports all clue types present in the dataset that can be evaluated against complete grids.

---

7. Step Count Statistics

Metric	Value
Average steps	**6.01**
Median steps	**6**
Minimum steps	**5**
Maximum steps	**7**

---

8. Discussion

The results demonstrate that a candidate-elimination-based symbolic CSP solver can efficiently solve a large subset of Zebra logic puzzles without search-heavy techniques or machine learning. Enumerating complete grids allows constraints to be applied deterministically and simplifies correctness reasoning.

The main limitation lies in natural language parsing: if a clue cannot be translated into a correct constraint check, valid solutions may be incorrectly discarded or retained. Additionally, puzzles with multiple valid solutions inherently reduce accuracy under single-solution evaluation.

---

9. Conclusion

This project presents a purely symbolic CSP solver for Zebra logic puzzles using exhaustive candidate generation and constraint-based elimination. Despite its conceptual simplicity, the solver achieves strong efficiency and competitive accuracy on a large benchmark dataset.

The clear separation between parsing, constraint checking, and evaluation makes the system easy to analyze, debug, and extend. Future work could explore improved clue parsing or hybrid approaches that combine candidate elimination with guided search.